Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO

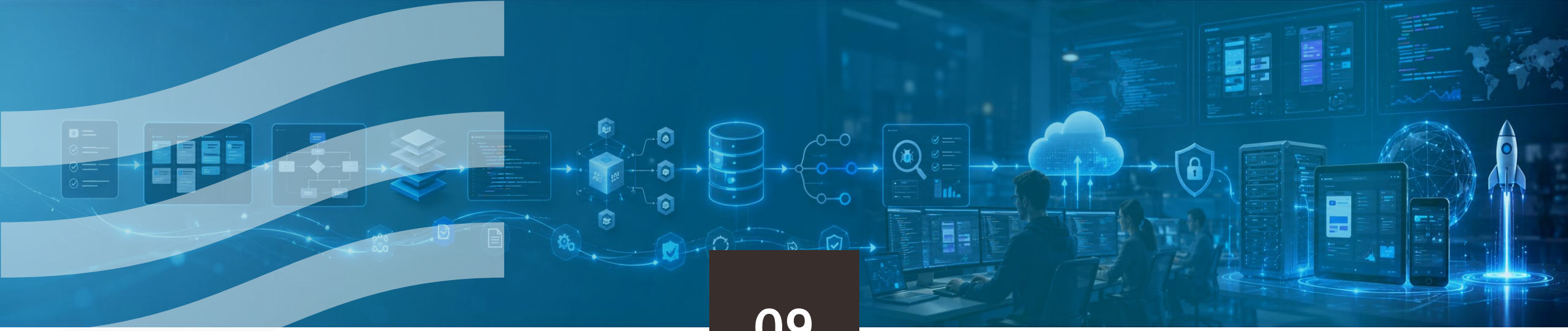


2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



09

Framework

Aula 33 a 36 - Desenvolvimento de Sistemas

Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

Entendendo o que é um Framework

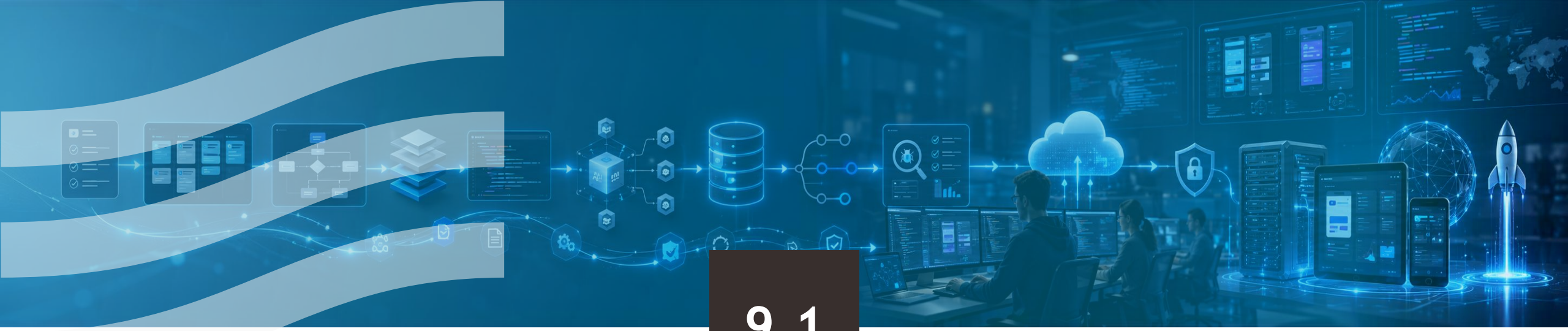
Um framework é uma estrutura pronta que auxilia o desenvolvedor na criação de aplicações. Ele fornece ferramentas, componentes e uma organização de código já definida, evitando que funcionalidades comuns precisem ser desenvolvidas do zero.Ao utilizar um framework, o programador escreve seu código seguindo regras e padrões estabelecidos por essa estrutura, tornando o desenvolvimento mais organizado, produtivo e fácil de manter.

Como um Framework ajuda no desenvolvimento

- Estrutura – Define uma forma organizada de separar e desenvolver as partes da aplicação.
- Recursos prontos – Oferece funcionalidades que podem ser reutilizadas, reduzindo a quantidade de código escrito do zero.
- Padronização – Estabelece regras e boas práticas para manter o projeto organizado.
- Produtividade – Permite que o desenvolvedor concentre maior esforço nas funcionalidades específicas da aplicação.

Exemplos de Frameworks

- Angular – Framework utilizado para desenvolver interfaces e aplicações web no Front-end.
- Spring Boot – Framework do ecossistema Java utilizado principalmente para aplicações Back-end e APIs.
- Laravel – Framework PHP utilizado no desenvolvimento de aplicações web.
- Django – Framework Python voltado para o desenvolvimento de aplicações web.



9.1

Spring Boot

Aula 33 a 36 - Desenvolvimento de Sistemas
Instrutor: Vitor Hugo Almeida de Sousa

Firjan SENAI

Spring Boot

O Spring Boot é um framework baseado no ecossistema Spring, utilizado principalmente para desenvolver aplicações Back-end com Java. Seu objetivo é simplificar a criação e a configuração de aplicações, fornecendo recursos prontos e configurações automáticas que evitam grande parte do trabalho manual necessário em projetos tradicionais.
Com o Spring Boot, o desenvolvedor pode criar aplicações web, APIs REST e sistemas conectados a bancos de dados de maneira mais rápida e organizada. Ele também possui servidor web incorporado, permitindo executar a aplicação diretamente, sem a necessidade de instalar e configurar separadamente um servidor externo.

Principais características

- Configuração automática – O Spring Boot identifica as dependências utilizadas no projeto e realiza diversas configurações automaticamente.
- Servidor integrado – Pode utilizar servidores como Tomcat diretamente dentro da aplicação.
- APIs REST – Facilita a criação de serviços que recebem e enviam dados pela web.
- Integração com banco de dados – Trabalha facilmente com JDBC, JPA e outros recursos de persistência.
- Organização do projeto – Permite separar responsabilidades em classes como Controller, Service, Repository e Model.

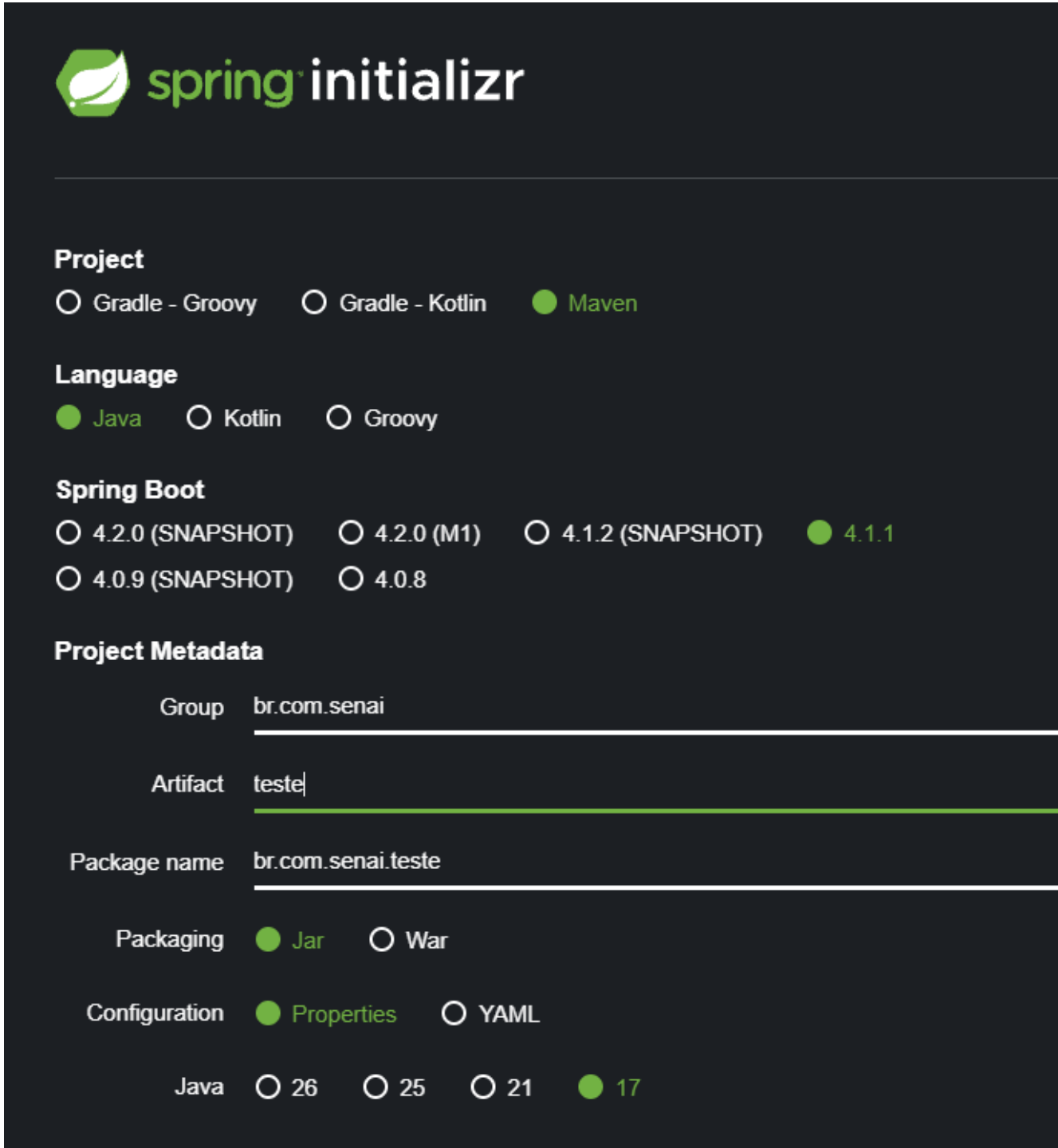
Spring Initializr

Primeiros passos com projetos Spring Boot: clique aqui

O Spring Initializr é uma ferramenta utilizada para gerar a estrutura inicial de um projeto Spring Boot. Com ele, podemos escolher a linguagem, a versão do Java, o nome do projeto e as dependências que serão utilizadas.

Isso evita a criação manual de toda a estrutura e já prepara o projeto para ser aberto em uma IDE, como o VS Code ou Eclipse.
Configurações iniciais:

- Project: Maven
- Language: Java
- Spring Boot: versão estável
- Group: identifica a organização ou pacote principal
- Artifact: nome do projeto
- Packaging: Jar
- Java: versão utilizada no ambiente



Spring Initializr

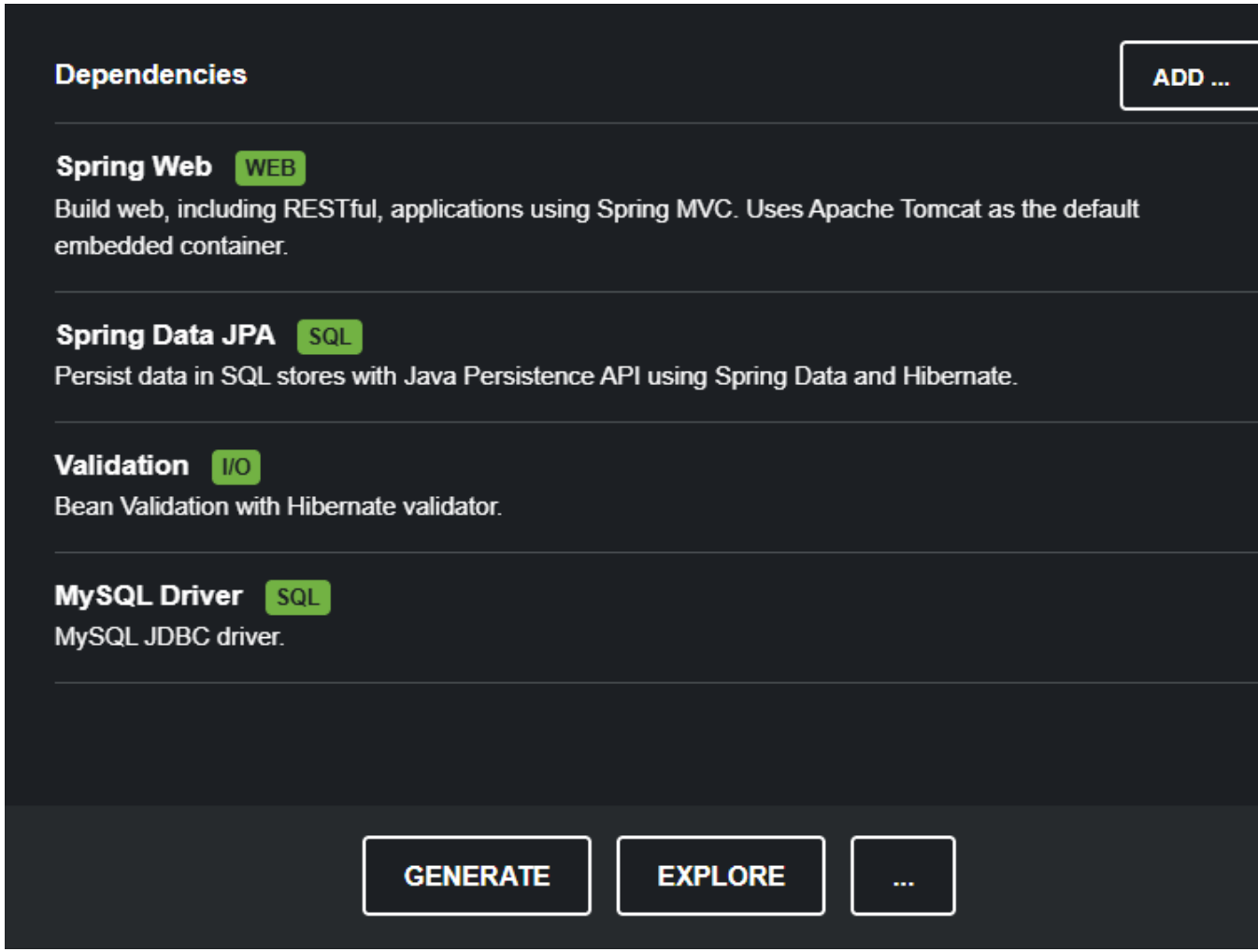
Escolhendo as dependências do projeto

Depois de preencher as informações básicas do projeto, o próximo passo é selecionar as **dependências** que serão utilizadas.

As dependências adicionam recursos prontos ao Spring Boot e evitam que diversas configurações sejam feitas manualmente.

Para este projeto, utilizaremos:

- Spring Web – permite criar aplicações web e APIs REST.
- Spring Data JPA – facilita o acesso e a manipulação de dados no banco.
- Validation – permite validar informações recebidas pela aplicação.
- MySQL Driver – realiza a conexão entre o projeto Java e o banco MySQL.



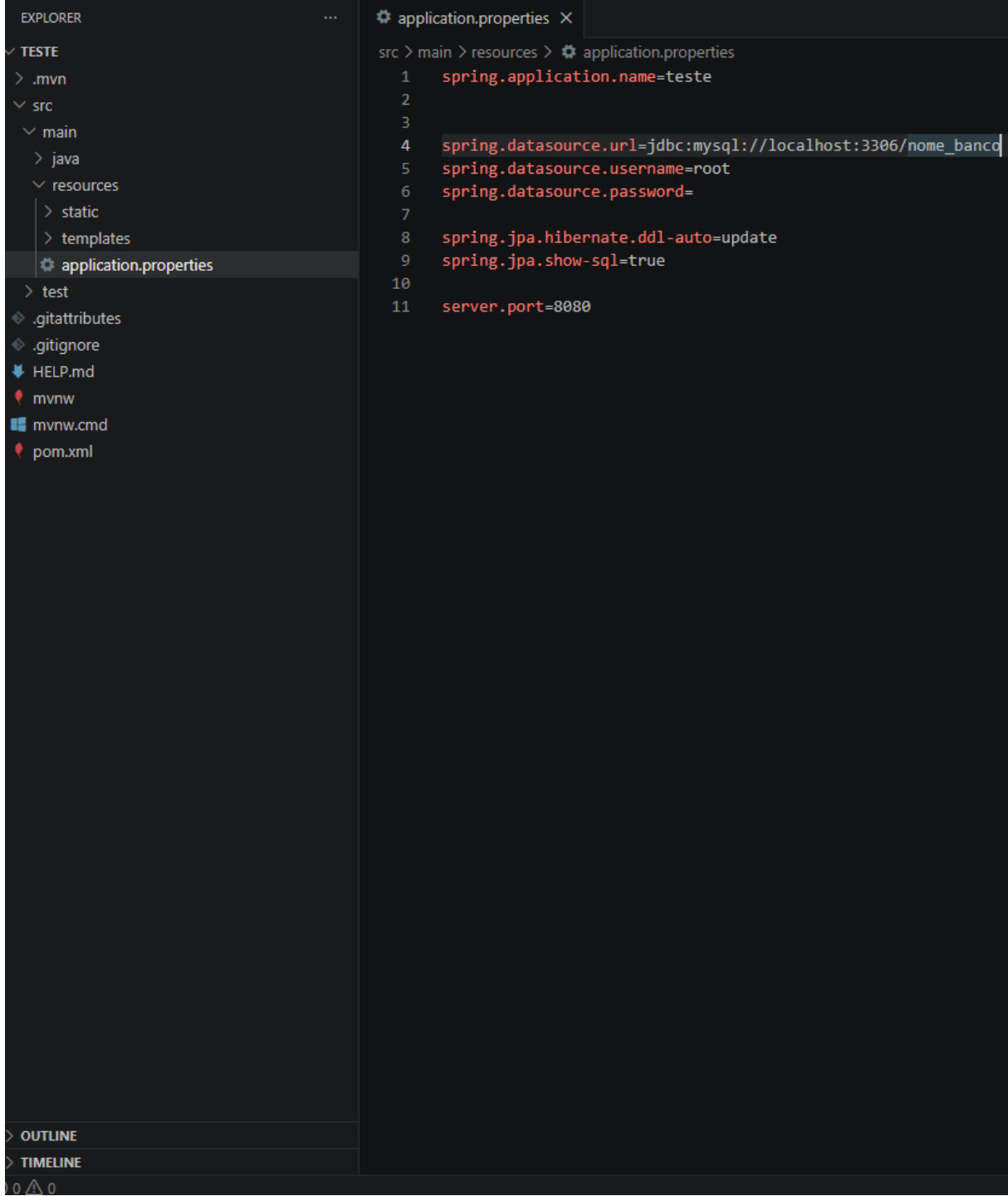
Configurando o Spring Boot

application.properties

O arquivo application.properties é utilizado para configurar o comportamento da aplicação Spring Boot sem precisar alterar diretamente o código Java.

Neste exemplo, ele define:

- spring.datasource.url – informa o endereço e o nome do banco de dados.
- spring.datasource.username – usuário utilizado para acessar o MySQL.
- spring.datasource.password – senha do banco.
- spring.jpa.hibernate.ddl-auto=update – permite que o Hibernate atualize a estrutura das tabelas conforme as classes do projeto.
- spring.jpa.show-sql=true – mostra no console os comandos SQL executados.
- server.port=8080 – define a porta em que a aplicação será executada.



Controller

Criando o primeiro Controller

Crie o pacote **controller** dentro do pacote principal do projeto. Em seguida, crie nele o arquivo **TesteController.java**.

Esse arquivo será responsável por receber uma requisição HTTP e devolver uma resposta simples.

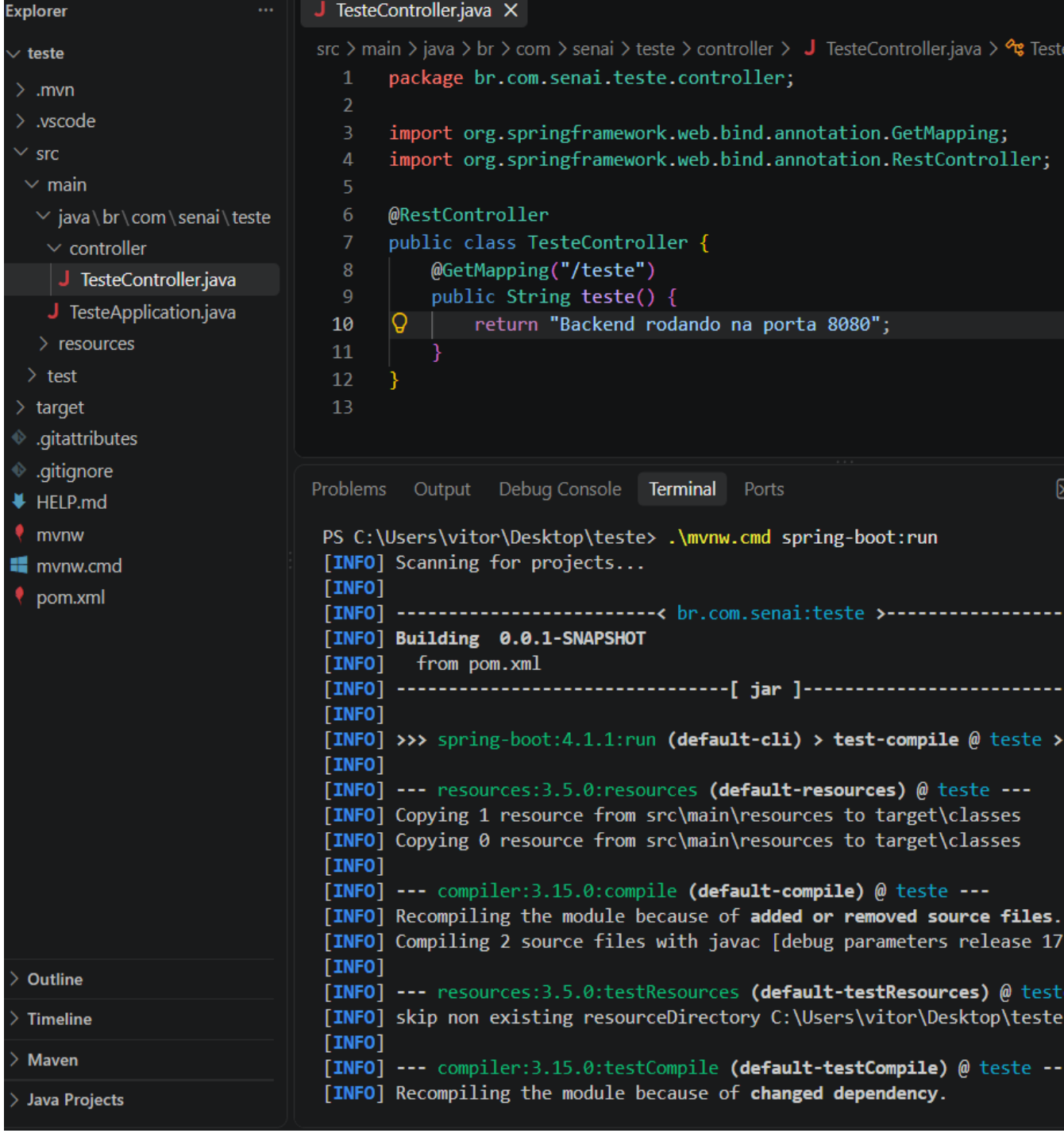
Para iniciar o projeto, abra o terminal na pasta principal e execute:

.\mvnw.cmd spring-boot:run

Aguarde a inicialização do servidor e acesse http://localhost:8080/teste no navegador.

Ele será usado para verificar se:

- o Spring Boot iniciou corretamente;
- o servidor está atendendo na porta configurada;
- a aplicação reconhece o endereço /teste;
- o Controller consegue retornar uma resposta ao navegador.



Controller: a porta de entrada do backend

O Controller é a camada responsável por receber as requisições HTTP enviadas pelo cliente, como um navegador ou aplicativo. Ele identifica qual operação foi solicitada, encaminha os dados para as outras camadas do sistema e devolve uma resposta HTTP.

Atenção



O Controller funciona como uma porta de entrada do backend. Ele define quais endereços estarão disponíveis, identifica o tipo de requisição recebida e determina qual método será executado.

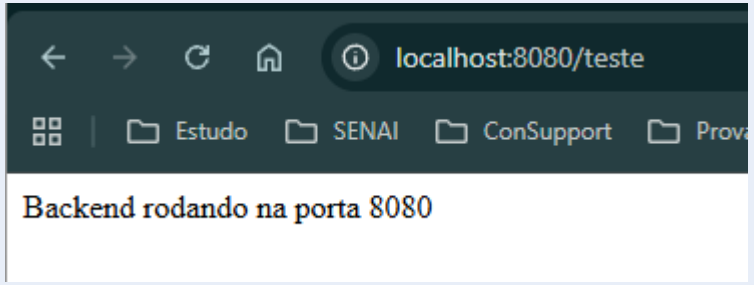
No exemplo criado, o endereço ***/teste*** está associado a uma requisição GET. Quando essa rota é acessada, o Controller executa o método testar() e devolve a mensagem configurada como resposta HTTP.

Testando requisições do backend

Depois de criar uma rota no Controller, é necessário testar se ela recebe corretamente a requisição HTTP e devolve a resposta esperada. Nesse teste, verificamos o método utilizado, o endereço da rota, o código de status e o conteúdo retornado pelo servidor.

Saiba Mais!

Ao acessar localhost:8080/teste, uma requisição GET é enviada ao backend. O Spring Boot localiza o método associado à rota /teste, executa o código e devolve a mensagem configurada. Se a resposta aparecer, a rota está funcionando corretamente.



Fique ON

Ferramentas como Postman, Insomnia e Bruno facilitam os testes de APIs. Elas permitem selecionar o método HTTP, informar a URL, enviar dados e analisar a resposta do servidor. Nas próximas práticas, utilizaremos o Postman para testar nossas requisições.

Nosso primeiro servidor em funcionamento

Até aqui, criamos e iniciamos uma aplicação **Spring Boot**, configuramos a porta do servidor e desenvolvemos um **Controller** com a rota **/teste**. Ao acessar essa rota, enviamos uma requisição HTTP ao backend e recebemos a resposta programada.

Resumo



Podemos imaginar o servidor como um estabelecimento que acabou de abrir suas portas. A porta 8080 é a entrada, o endereço /teste indica o setor procurado e o Controller funciona como o atendente: recebe o pedido, identifica o que foi solicitado e entrega uma resposta.

Por enquanto, o acesso foi feito no próprio computador por meio do localhost. Futuramente, um frontend ou outro sistema poderá fazer requisições ao backend para consultar, cadastrar, atualizar ou excluir informações.

Postman

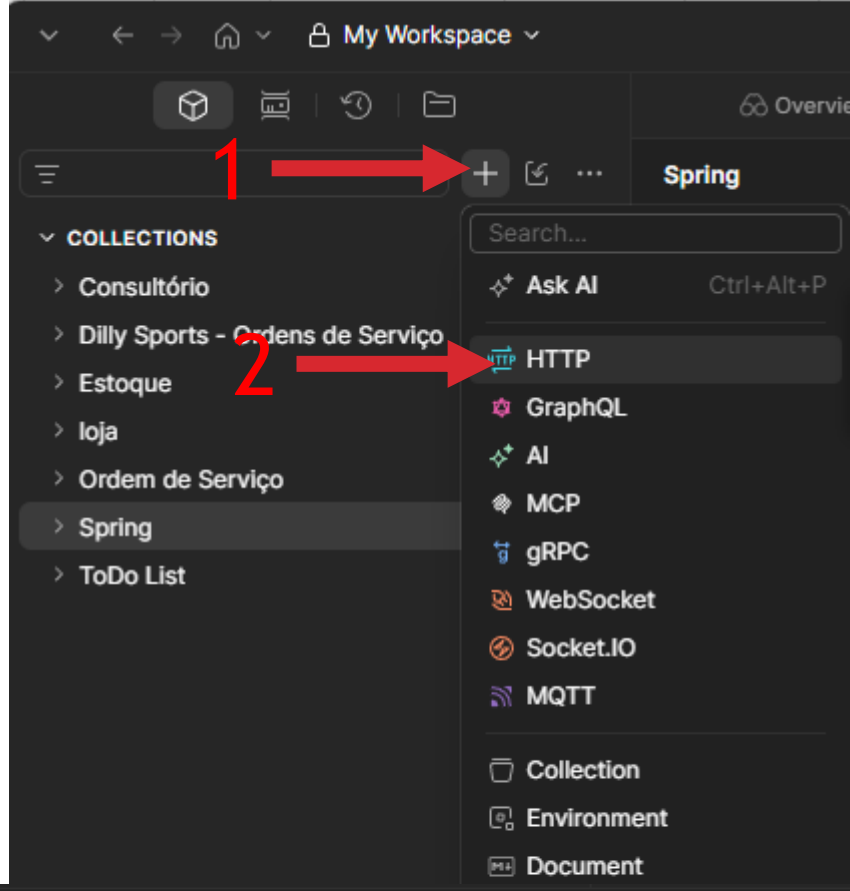
Testando a requisição com o Postman

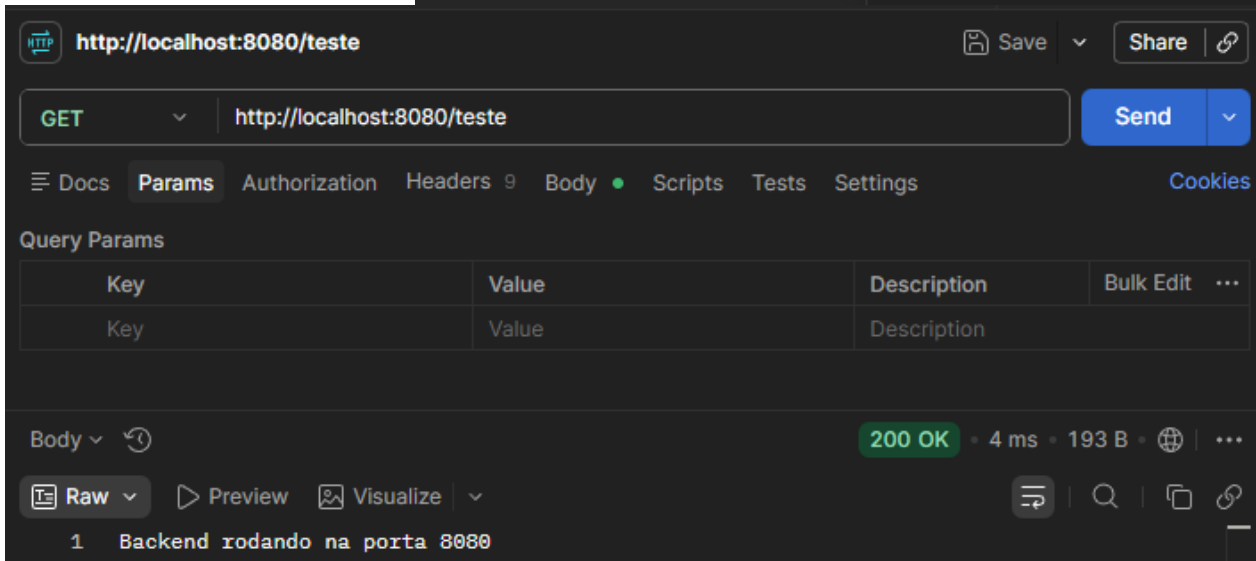
O **Postman** é uma ferramenta utilizada para enviar requisições HTTP e verificar as respostas de uma API. Ele permite testar o backend diretamente, mesmo antes da criação de uma interface gráfica ou frontend.

Com o servidor Spring Boot em execução:

- Clique no botão + indicado na imagem.
- Selecione a opção HTTP.
- Escolha o método GET.
- Digite http://localhost:8080/teste.
- Clique em Send.

*Na parte inferior, verifique o código **200 OK** e a mensagem retornada pelo Controller. Esses resultados indicam que a requisição foi recebida e respondida corretamente.*





Model

O Model define os dados que representam algo no sistema, como um aluno, um livro ou um equipamento.

Imagine uma ficha de aluno na secretaria escolar. Ela possui campos como identificação, nome e e-mail. No sistema, a classe Aluno representa essa ficha, e seus atributos representam os campos.

Cada objeto Aluno contém os dados de um aluno específico. Neste projeto, as anotações JPA também relacionam essa classe com uma tabela do banco:

- @Entity: identifica uma entidade que poderá ser armazenada.
- @Table: define a tabela correspondente.
- @Id: identifica a chave primária.
- @GeneratedValue: configura a geração automática do ID.

Model

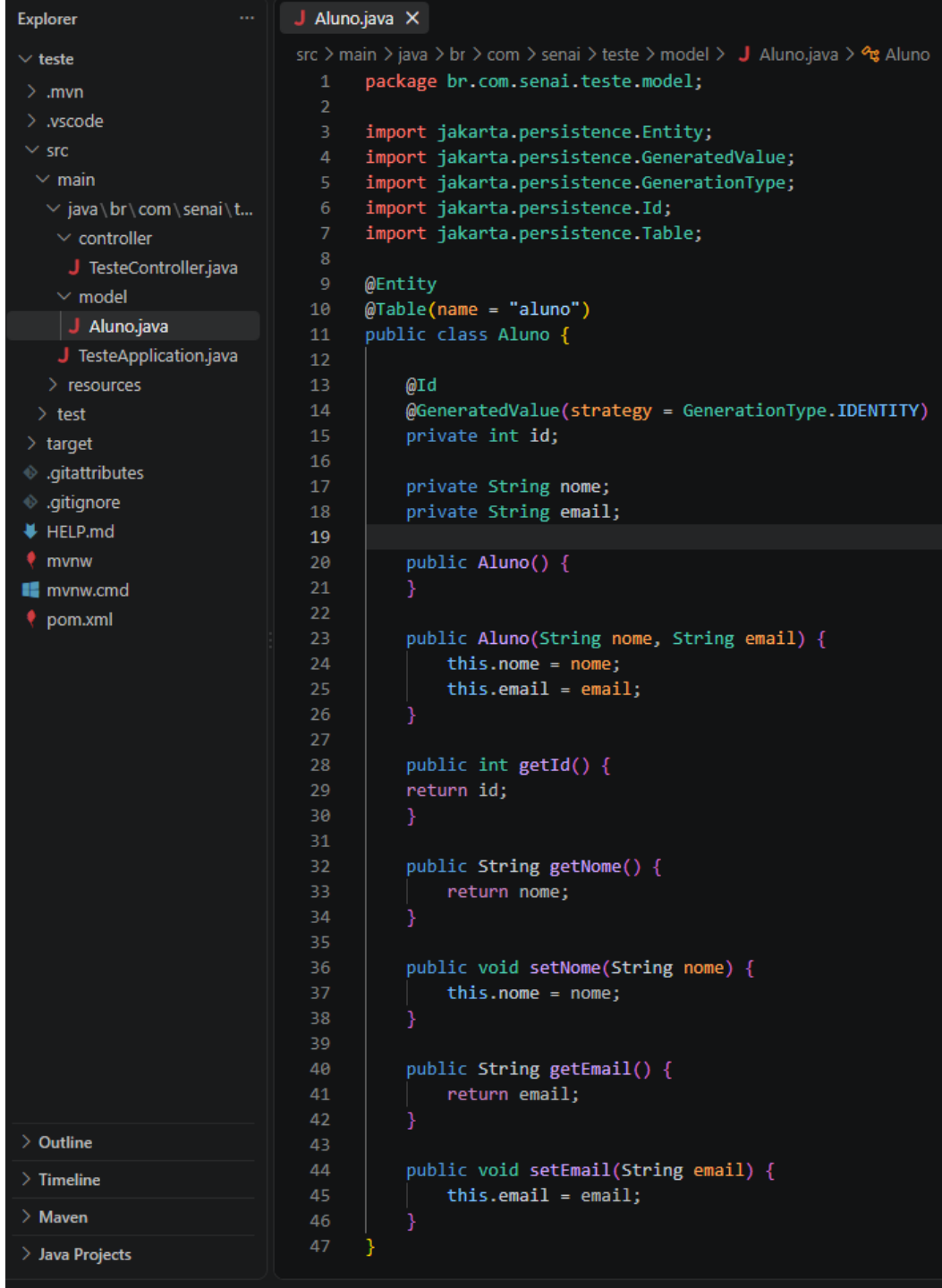
Representando o aluno no Spring Boot

O primeiro passo será criar uma classe Java para representar os dados do aluno. Essa classe será utilizada para receber, transportar e armazenar as informações durante o ciclo da requisição.

Crie o pacote model dentro do pacote principal do projeto. Em seguida, crie o arquivo Aluno.java.

A classe Aluno funciona como uma representação, em Java, da tabela aluno existente no banco de dados.

- @Entity informa que a classe representa uma entidade que poderá ser armazenada no banco.
- @Table(name = "aluno") relaciona a classe com a tabela aluno.
- @Id identifica o atributo correspondente à chave primária.
- @GeneratedValue informa que o ID será gerado automaticamente pelo MySQL.
- nome e email representam as demais colunas utilizadas no cadastro.



JPA

JPA é a sigla para Java Persistence API. No Spring, ela é usada para facilitar a comunicação entre a aplicação Java e um banco de dados relacional.

Com JDBC, você faz muita coisa manualmente:

```
String sql = "INSERT INTO aluno (nome, email) VALUES (?, ?)";

PreparedStatement stmt = conexao.prepareStatement(sql);

stmt.setString(1, aluno.getNome());
stmt.setString(2, aluno.getEmail());

stmt.executeUpdate();
```

Saiba Mais!



Com JPA, você trabalha mais diretamente com objetos Java:

```
Aluno aluno = new Aluno();

aluno.setNome("Carlos");
aluno.setEmail("carlos@email.com");

alunoRepository.save(aluno);
```

Modelando os dados do aluno na aplicação

Até aqui, já temos o servidor Spring Boot funcionando e uma rota de teste criada. Agora, o próximo passo é começar a representar os dados que a aplicação irá receber e utilizar. Para isso, criamos a classe Aluno, que funcionará como o Model responsável por representar, dentro do Java, as informações de um aluno, como nome e e-mail.

Fique ON



O construtor vazio é necessário para que o Spring e o JPA consigam criar objetos da classe automaticamente ao receber dados de uma requisição ou consultar o banco. O segundo construtor facilita a criação manual de um aluno quando já conhecemos seu nome e e-mail.

Resumo



O Model funciona como uma ficha de aluno dentro da aplicação. Essa ficha possui os mesmos dados principais encontrados na tabela do banco. Quando o Postman enviar um nome e um e-mail, o Spring utilizará essas informações para preencher um objeto Aluno.

Repository

O Repository é a camada responsável pelo acesso ao banco de dados. Por meio dele, a aplicação pode salvar, consultar, atualizar e excluir registros.

Na analogia da secretaria escolar, ele funciona como o responsável pelo arquivo: guarda as fichas, encontra os registros solicitados e realiza alterações quando necessário.

O Service solicita uma operação, e o Repository fornece o acesso aos dados para realizá-la.

Ao estender JpaRepository, o AlunoRepository recebe operações prontas, como:

- save(): salvar ou atualizar um aluno.
- findAll(): consultar todos os alunos.
- findById(): buscar um aluno pelo ID.
- deleteById(): excluir um aluno pelo ID.

Repository

Acesso aos dados com Spring Data JPA

No projeto com JDBC, a classe AlunoDAO concentrava os comandos SQL e as operações realizadas na tabela aluno.

Com o Spring Data JPA, criaremos um Repository. Ele fornecerá operações prontas de cadastro, consulta, atualização e exclusão, reduzindo a necessidade de escrever manualmente comandos SQL e métodos repetitivos.

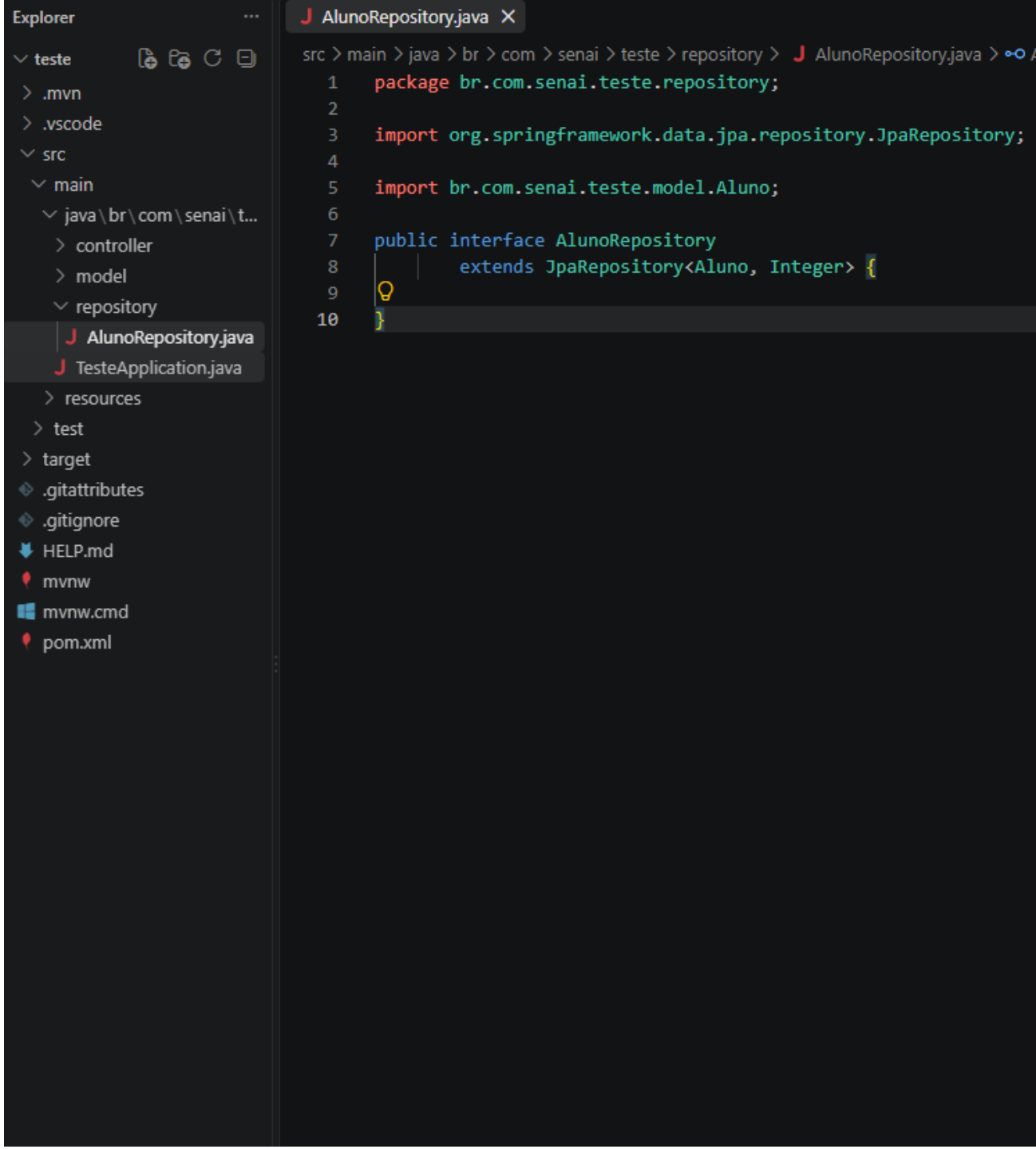
Dentro do pacote principal do projeto:

- Crie o pacote repository.
- Dentro dele, crie a interface AlunoRepository.
- Adicione o código ao lado

AlunoRepository estende JpaRepository, uma interface fornecida pelo Spring Data JPA.

Os tipos informados entre < > identificam:

- Aluno: entidade que será manipulada;
- Integer: tipo utilizado pelo atributo id.



Service

O Service é a camada responsável por organizar as operações da aplicação e aplicar as regras de negócio, ou seja, as condições que o sistema deve respeitar.

Na analogia da secretaria escolar, ele funciona como o responsável por analisar uma solicitação. Antes de autorizar um cadastro, por exemplo, ele verifica se as condições necessárias foram atendidas.

O Controller recebe o pedido e o encaminha ao Service. O Service verifica as regras e utiliza o Repository quando precisa consultar ou salvar dados.

No cadastro de um aluno, essa camada pode:

- Verificar se o e-mail já está cadastrado.
- Impedir o cadastro quando uma regra não for atendida.
- Solicitar ao Repository que salve o aluno.
- Devolver o resultado ao Controller.

Service

Organizando a operação de cadastro

O Service será responsável por coordenar a operação de cadastro. Ele receberá o objeto Aluno enviado pelo Controller e utilizará o AlunoRepository para gravar os dados no banco.

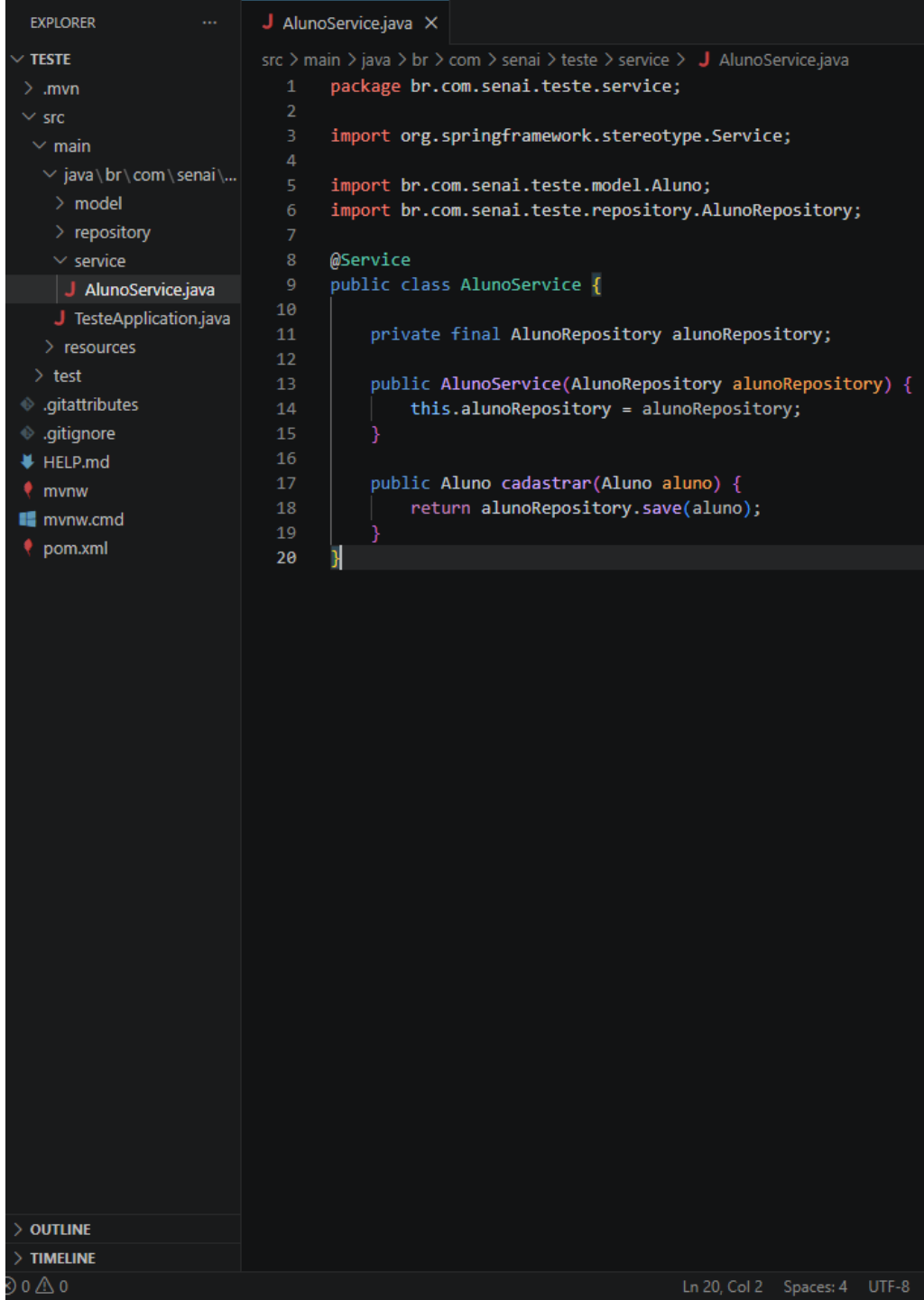
Neste primeiro exemplo, o Service terá somente o método necessário para cadastrar um aluno.

Dentro do pacote principal do projeto:

- Crie o pacote service.
- Dentro dele, crie a classe AlunoService.
- Adicione o código ao lado

A anotação @Service informa ao Spring que essa classe representa uma camada responsável pelas operações da aplicação.

O AlunoRepository é recebido pelo construtor. Durante a inicialização, o próprio Spring fornece o Repository necessário para criar o AlunoService. Esse processo é chamado de injeção de dependência.



Detalhando Service

O método save() grava os dados na tabela e devolve o objeto após o cadastro, agora contendo o ID gerado pelo banco.

Saiba Mais!



A injeção de dependência permite utilizar objetos administrados pelo Spring sem criá-los manualmente com new. Assim, a aplicação pode receber uma implementação pronta do AlunoRepository.

Service → Repository → Banco de dados

Controller

O Controller é a porta de entrada da API. Ele define as rotas, recebe as requisições HTTP e devolve as respostas para quem acessou o sistema.

Na secretaria escolar, ele funciona como o atendente: recebe a solicitação, encaminha ao responsável pelo serviço e comunica o resultado.

Neste projeto, quando o cliente envia uma requisição POST para /alunos, o Controller:

- Recebe os dados enviados em JSON.
- Converte esses dados em um objeto Aluno.
- Solicita o cadastro ao AlunoService.
- Devolve o resultado em uma resposta HTTP.

Controller

Recebendo a requisição de cadastro

O Controller disponibilizará o endereço utilizado para cadastrar alunos. Quando uma requisição POST for enviada para /alunos, ele receberá os dados, solicitará o cadastro ao Service e devolverá o aluno cadastrado.

Crie o arquivo AlunoController.java dentro do pacote controller.

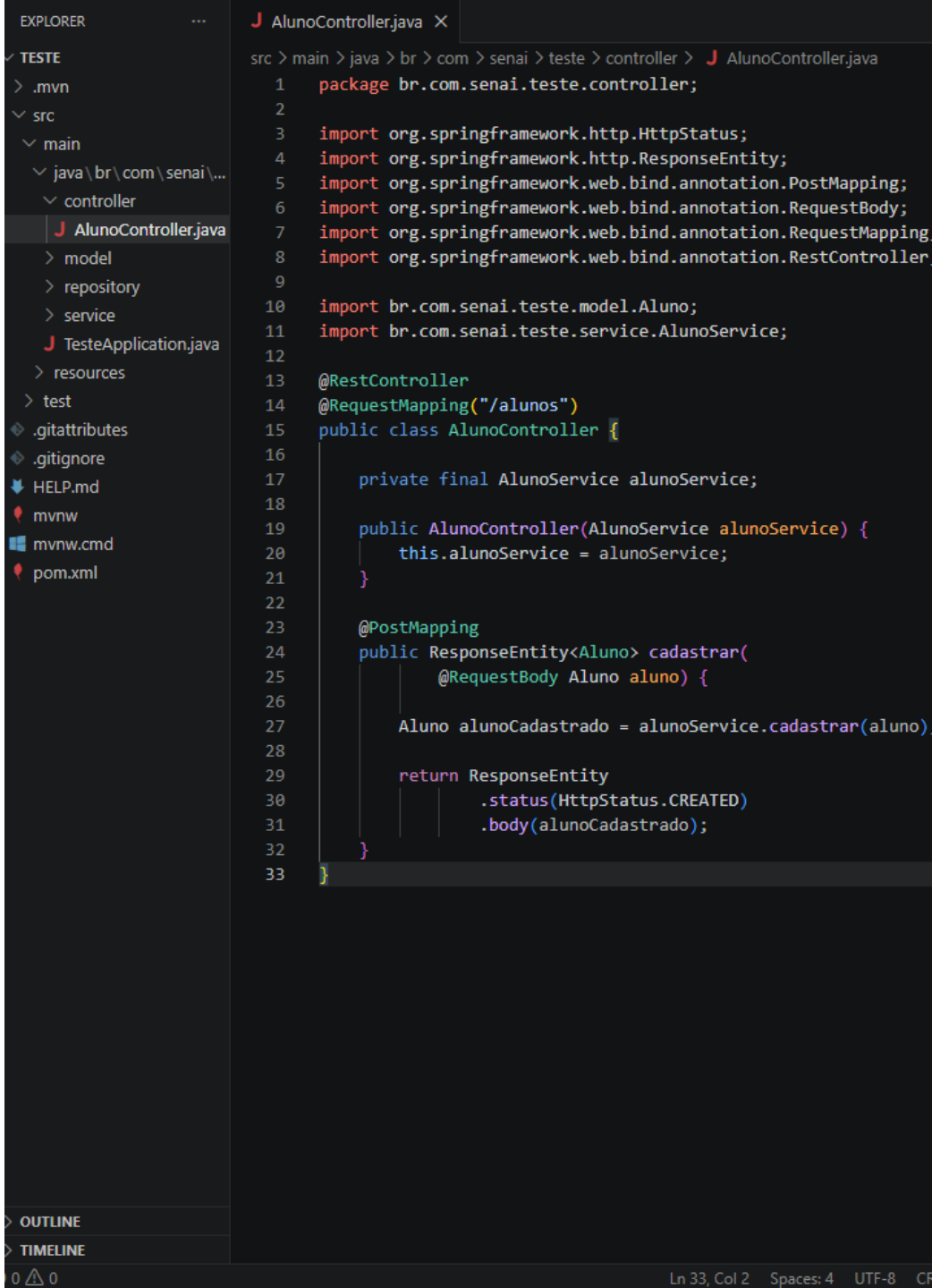
O Controller recebe a requisição HTTP e encaminha o objeto Aluno para o Service.

Após o cadastro, o método devolve:

- o código HTTP 201 Created;
- os dados do aluno cadastrado;
- o ID gerado pelo banco.

AlunoRepository estende JpaRepository, uma interface fornecida pelo Spring Data JPA.

O AlunoService é recebido pelo construtor. Assim como aconteceu com o Repository, o Spring fornece automaticamente o objeto necessário.



Firjan SENAI

Controller

Anotações do Controller

As anotações informam ao Spring como a classe, o método e os dados recebidos deverão ser utilizados durante uma requisição.

- **@RestController**

É colocada acima da classe para informar que ela receberá requisições HTTP e devolverá dados diretamente na resposta. Os objetos Java retornados são convertidos automaticamente para JSON.

- **@RequestMapping("/alunos")**

Define o endereço principal do Controller. Todas as rotas criadas nessa classe começarão com /alunos.

- **@PostMapping**

É colocada acima do método para associá-lo a uma requisição HTTP do tipo POST.

Neste caso, o método será executado quando o backend receber:

- **POST** http://localhost:8080/alunos

- **@RequestBody**

Informa que os dados enviados no corpo da requisição devem ser convertidos em um objeto Aluno.

AlunoController.java

src > main > java > br > com > senai > teste > controller > AlunoController.java

```java
package br.com.senai.teste.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.service.AlunoService;

@RestController
@RequestMapping("/alunos")
public class AlunoController {

    private final AlunoService alunoService;

    public AlunoController(AlunoService alunoService) {
        this.alunoService = alunoService;
    }

    @PostMapping
    public ResponseEntity<Aluno> cadastrar(
            @RequestBody Aluno aluno) {

        Aluno alunoCadastrado = alunoService.cadastrar(aluno);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(alunoCadastrado);
    }
}
```

Ln 33, Col 2 Spaces: 4 UTF-8

Quem chama esse construtor e quem cria o AlunoService?

```
public AlunoController(AlunoService alunoService) {
    this.alunoService = alunoService;
}
```

A resposta é: o próprio Spring.

Saiba Mais!



Quando sua aplicação inicia, o Spring procura classes que ele deve administrar. Por exemplo:

```
@RestController
public class AlunoController {
```

e, no Service, provavelmente:

```
@Service
public class AlunoService {
```

O Spring cria esses objetos.Conceitualmente, é como se o Spring fizesse isto por você:

```
AlunoService service = new AlunoService(...);

AlunoController controller =
        new AlunoController(service);
```

@RestController

Atenção



@RestController configura a classe inteira como um Controller REST. Já @PostMapping configura apenas o método responsável pelo cadastro. Por isso, uma anotação aparece acima da classe e a outra acima do método.

Ciclo construído
Postman → Controller → Service → Repository → MySQL

Firjan SENAI

JSON

Como os dados são representados nas requisições

JSON significa **JavaScript Object Notation**. É um formato de texto utilizado para representar e transportar dados de maneira estruturada entre sistemas.

Em aplicações web, ele é muito utilizado na comunicação entre **frontend e backend**. Quando um cliente faz uma requisição para uma API, os dados podem ser enviados em JSON. Da mesma forma, o servidor pode responder utilizando esse formato.

O JSON organiza as informações em pares de **chave e valor**, o que facilita a leitura tanto por pessoas quanto por aplicações.

No nosso projeto, um aluno possui informações como **id, nome e email**. Em uma requisição HTTP, esses dados podem ser representados em JSON.

```
{
  "id": 1,
  "nome": "João Silva",
  "email": "joao@email.com"
}
```

Postman

Enviando uma requisição POST

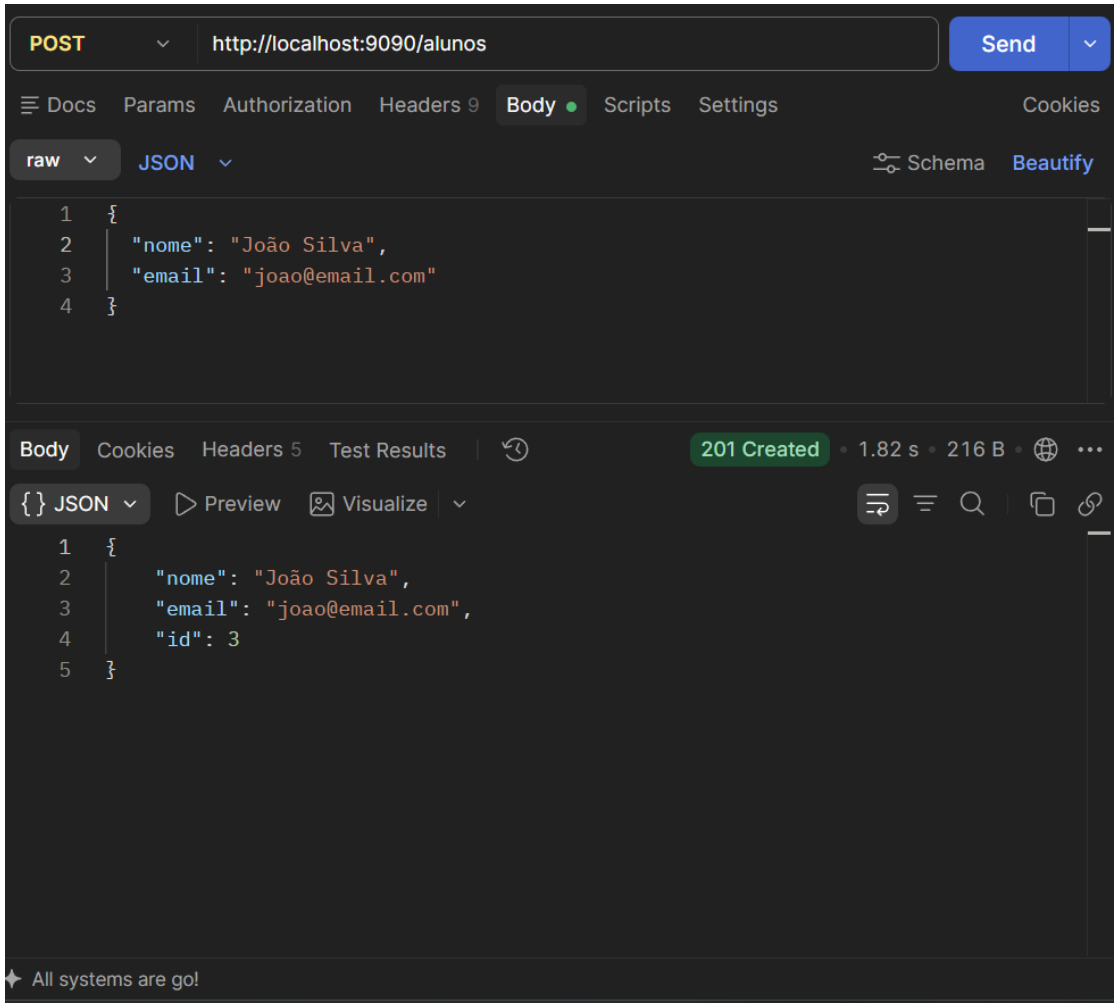
Com as camadas do cadastro concluídas, podemos testar o ciclo completo. O Postman enviará os dados do aluno para o Controller, que encaminhará a operação até o banco de dados.

Antes do teste, confirme que o MySQL está em execução e inicie o projeto Spring Boot.

No Postman:

- Crie uma nova requisição HTTP.
- Selecione o método POST.
- Digite o endereço:
  - http://localhost:8080/alunos
- Acesse a aba Body.
- Selecione raw.
- Escolha o formato JSON.
- Insira os dados

O servidor deverá responder com o código:
201 Created



Caminho da requisição

A requisição leva os dados do cliente até o banco. A resposta confirma o resultado da operação e devolve ao cliente os dados do recurso criado.

Resumo



Postman → Controller → Service → Repository → MySQL
Postman ← Controller ← Service ← Repository ← MySQL

Firjan SENAI

Consulta de dados

Criando a requisição GET

Depois de cadastrar um aluno, o próximo passo será consultar os registros armazenados. Para isso, adicionaremos uma operação que utiliza o método findAll() fornecido pelo JpaRepository.

A requisição será enviada para o mesmo endereço /alunos, mas utilizará o método HTTP GET.

O método findAll() consulta todos os registros da tabela aluno e devolve uma lista de objetos Aluno.

Como essa operação já é fornecida pelo JpaRepository, não precisamos criar manualmente o comando SELECT.

AlunoService.java

src > main > java > br > com > senai > teste > service > AlunoService.java

```java
package br.com.senai.teste.service;

import org.springframework.stereotype.Service;
import java.util.List;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.repository.AlunoRepository;

@Service
public class AlunoService {

    private final AlunoRepository alunoRepository;

    public AlunoService(AlunoRepository alunoRepository) {
        this.alunoRepository = alunoRepository;
    }

    public Aluno cadastrar(Aluno aluno) {
        return alunoRepository.save(aluno);
    }

    public List<Aluno> listar() {
        return alunoRepository.findAll();
    }
}
```

Alteração no AlunoController

Criando a requisição GET

Dentro do controller:

- Adicione as importações
- Adicione o novo método dentro da classe

A anotação @GetMapping associa o método listar() a uma requisição HTTP do tipo GET.

A rota completa será:

- GET http://localhost:8080/alunos

O método ResponseEntity.ok() devolve a lista com o código HTTP:

- 200 OK

AlunoController.java

src > main > java > br > com > senai > teste > controller > AlunoController.java

```java
package br.com.senai.teste.controller;

import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.GetMapping;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.service.AlunoService;

@RestController
@RequestMapping("/alunos")
public class AlunoController {

    private final AlunoService alunoService;

    public AlunoController(AlunoService alunoService) {
        this.alunoService = alunoService;
    }

    @PostMapping
    public ResponseEntity<Aluno> cadastrar(
            @RequestBody Aluno aluno) {

        Aluno alunoCadastrado = alunoService.cadastrar(aluno);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(alunoCadastrado);
    }

    @GetMapping
    public ResponseEntity<List<Aluno>> listar() {

        List<Aluno> alunos = alunoService.listar();

        return ResponseEntity.ok(alunos);
    }
}
```

Postman

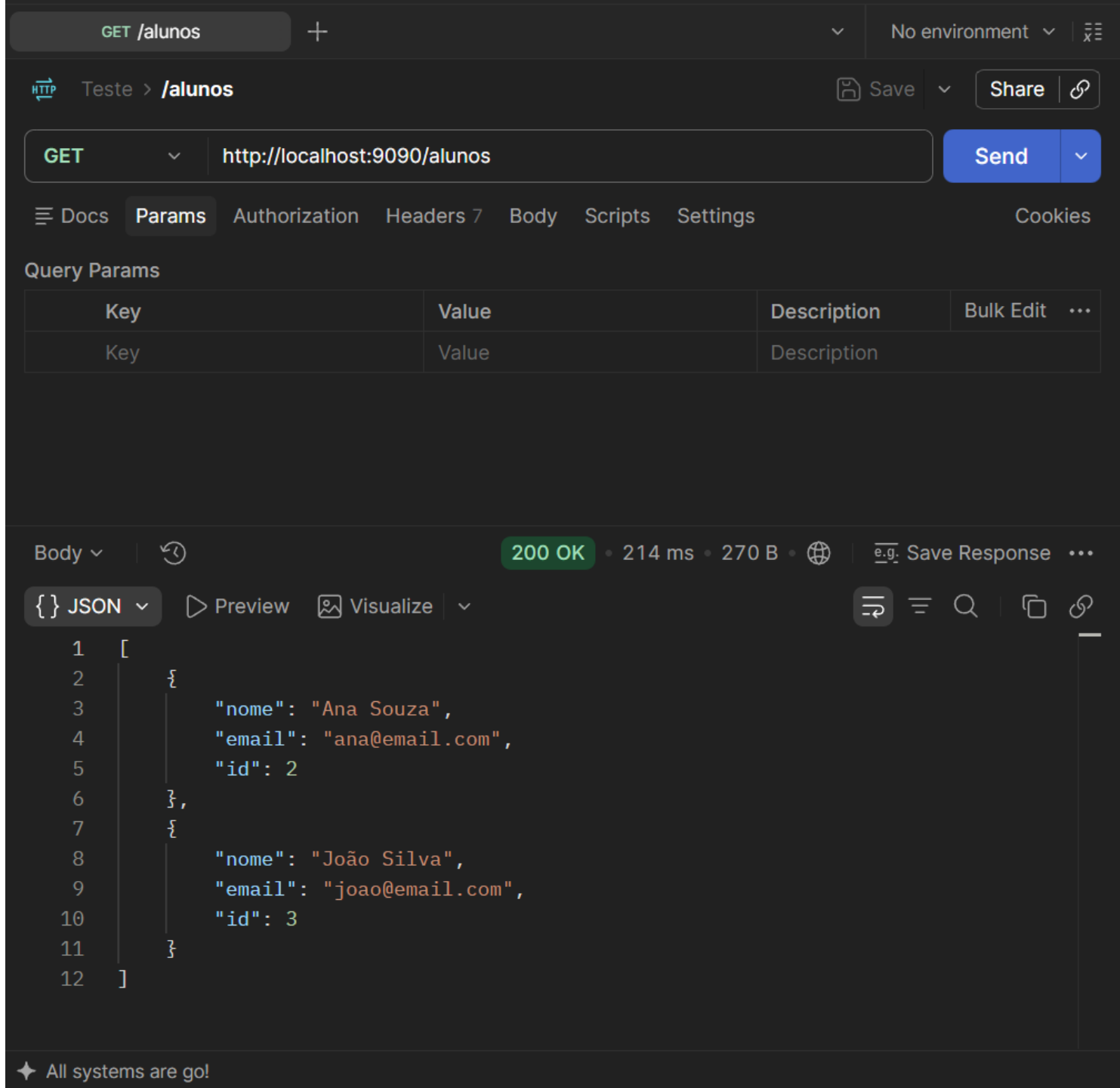
Enviando uma requisição GET

Com a operação de consulta criada, utilizaremos o Postman para enviar uma requisição GET e verificar os alunos armazenados no banco.

No Postman:

- Mantenha a aplicação Spring Boot em execução.
- Crie uma nova requisição no Postman.
- Selecione o método GET.
- Digite o endereço:
- http://localhost:8080/alunos
- Não preencha a aba Body.
- Clique em Send.

Resultado esperado
O servidor deverá responder com:
200 OK



Getters e o retorno da API

Quando o Spring precisa transformar um objeto Java em JSON, ele utiliza os métodos getters da classe para acessar os valores que serão enviados na resposta.

Fique ON



Os getters permitem que o Spring/Jackson acesse os dados do objeto durante a criação do JSON. Por isso, se um atributo não possuir um método get, ele pode não aparecer no retorno da API, mesmo existindo dentro da classe.

Consulta por ID

Localizando um aluno específico

ALTERAÇÃO NO AlunoService:

- Adicione a importação
- Adicione o método

A listagem devolve todos os alunos cadastrados. Agora, criaremos uma rota que recebe um ID e consulta somente o aluno correspondente.

O endereço utilizado seguirá o formato

- GET http://localhost:8080/alunos/1

O método findById() realiza a consulta pela chave primária. O resultado é um Optional, pois o ID informado pode existir ou não no banco.

AlunoService.java

src > main > java > br > com > senai > teste > service > AlunoService.java > Aluno

```java
package br.com.senai.teste.service;

import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.repository.AlunoRepository;

@Service
public class AlunoService {

    private final AlunoRepository alunoRepository;

    public AlunoService(AlunoRepository alunoRepository) {
        this.alunoRepository = alunoRepository;
    }

    public Aluno cadastrar(Aluno aluno) {
        return alunoRepository.save(aluno);
    }

    public List<Aluno> listar() {
        return alunoRepository.findAll();
    }

    public Optional<Aluno> buscarPorId(Integer id) {
        return alunoRepository.findById(id);
    }
}
```

Consulta por ID

Localizando um aluno específico

ALTERAÇÃO NO AlunoService:

- Adicione a importação
- Adicione o método

A listagem devolve todos os alunos cadastrados. Agora, criaremos uma rota que recebe um ID e consulta somente o aluno correspondente.

O endereço utilizado seguirá o formato

- GET http://localhost:8080/alunos/1

O método findById() realiza a consulta pela chave primária. O resultado é um Optional, pois o ID informado pode existir ou não no banco.

AlunoService.java

src > main > java > br > com > senai > teste > service > AlunoService.java > Aluno

```java
package br.com.senai.teste.service;

import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.repository.AlunoRepository;

@Service
public class AlunoService {

    private final AlunoRepository alunoRepository;

    public AlunoService(AlunoRepository alunoRepository) {
        this.alunoRepository = alunoRepository;
    }

    public Aluno cadastrar(Aluno aluno) {
        return alunoRepository.save(aluno);
    }

    public List<Aluno> listar() {
        return alunoRepository.findAll();
    }

    public Optional<Aluno> buscarPorId(Integer id) {
        return alunoRepository.findById(id);
    }
}
```

Consulta por ID

Localizando um aluno específico

ALTERAÇÃO NO AlunoController:

- Adicione a importação
- Adicione o método

A anotação @PathVariable captura um valor presente no endereço da requisição. Ao acessar /alunos/1, o número 1 é recebido pelo parâmetro id e utilizado na consulta.

O Optional indica que o resultado pode ou não conter um aluno. Se o registro for encontrado, a API devolve 200 OK. Caso o ID não exista, devolve 404 Not Found.

AlunoController.java

src > main > java > br > com > senai > teste > controller > AlunoController.java > Alu

```
19  public class AlunoController {
27      @PostMapping
28      public ResponseEntity<Aluno> cadastrar(
29              @RequestBody Aluno aluno) {
30
31          Aluno alunoCadastrado = alunoService.cadastrar(aluno);
32
33          return ResponseEntity
34                  .status(HttpStatus.CREATED)
35                  .body(alunoCadastrado);
36      }
37
38      @GetMapping
39      public ResponseEntity<List<Aluno>> listar() {
40
41          List<Aluno> alunos = alunoService.listar();
42
43          return ResponseEntity.ok(alunos);
44      }
45
46      @GetMapping("/{id}")
47      public ResponseEntity<Aluno> buscarPorId(
48              @PathVariable Integer id) {
49
50          Optional<Aluno> aluno = alunoService.buscarPorId(id);
51
52          if (aluno.isPresent()) {
53              return ResponseEntity.ok(aluno.get());
54          }
55
56          return ResponseEntity.notFound().build();
57      }
58  }
```

Postman

Consultando um aluno específico

A rota de consulta por ID pode produzir dois resultados: devolver o aluno encontrado ou informar que não existe um cadastro com o identificador recebido.

Realize os dois testes para verificar o comportamento da API.

TESTE 1 — ID EXISTENTE
No Postman:

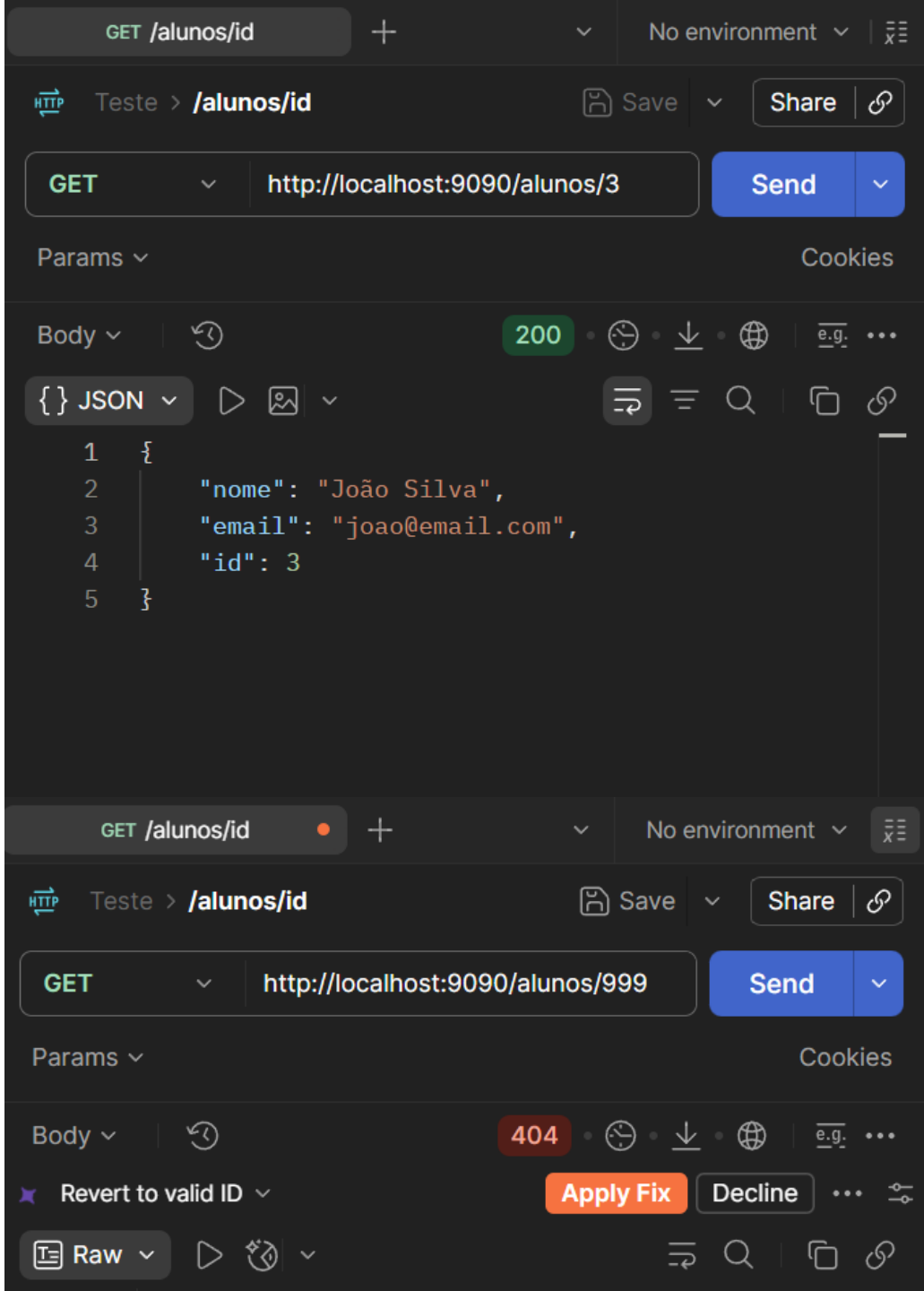
- Selecione o método GET;
- informe um ID existente no endereço;
- não envie conteúdo no Body;
- clique em Send.

Resultado esperado: 200 OK

TESTE 2 — ID INEXISTENTE
No Postman:

- Altere o endereço para um ID que não esteja cadastrado:
  - http://localhost:8080/alunos/999

Resultado esperado: 404 Not Found



Atualização de aluno

Localizando e modificando o cadastro

Antes de atualizar um aluno, a aplicação precisa verificar se o ID informado existe. Quando o registro é encontrado, os novos dados são aplicados ao objeto e enviados novamente ao Repository.

ALTERAÇÃO NO AlunoService:

- Adicione a importação
- Adicione o método

O método começa procurando o aluno pelo ID. Se nenhum registro for encontrado, devolve um Optional vazio.

Quando o aluno existe, o nome e o e-mail são substituídos pelos novos valores. Em seguida, save() grava as alterações e devolve o objeto atualizado.

AlunoService.java

src > main > java > br > com > senai > teste > service > AlunoService.java > AlunoService

```
11  public class AlunoService {
17      }
18
19      public Aluno cadastrar(Aluno aluno) {
20          return alunoRepository.save(aluno);
21      }
22
23      public List<Aluno> listar() {
24          return alunoRepository.findAll();
25      }
26
27      public Optional<Aluno> buscarPorId(Integer id) {
28          return alunoRepository.findById(id);
29      }
30
31      public Optional<Aluno> atualizar(
32              Integer id, Aluno novosDados) {
33
34          Optional<Aluno> alunoEncontrado = alunoRepository.findById(id);
35
36          if (alunoEncontrado.isEmpty()) {
37              return Optional.empty();
38          }
39
40          Aluno aluno = alunoEncontrado.get();
41
42          aluno.setNome(novosDados.getNome());
43          aluno.setEmail(novosDados.getEmail());
44
45          return Optional.of(alunoRepository.save(aluno));
46      }
47  }
```

Atualização de aluno

Recebendo uma requisição PUT

A rota de atualização receberá o ID pelo endereço e os novos dados pelo corpo da requisição. O Controller encaminhará essas informações ao Service e devolverá uma resposta conforme o resultado da operação.

ALTERAÇÃO NO AlunoController:

- Adicione a importação
- Adicione o método

O @PutMapping("/{id}") associa o método a uma requisição PUT que contém o ID no endereço.

O @PathVariable recebe o ID da rota, enquanto o @RequestBody transforma o JSON enviado em um objeto Aluno.

AlunoController.java

src > main > java > br > com > senai > teste > controller > AlunoController.java > ...

```java
@RestController
@RequestMapping("/alunos")
public class AlunoController {

    private final AlunoService alunoService;

    public AlunoController(AlunoService alunoService) {...

    @PostMapping
    public ResponseEntity<Aluno> cadastrar(...

    @GetMapping
    public ResponseEntity<List<Aluno>> listar() {...

    @GetMapping("/{id}")
    public ResponseEntity<Aluno> buscarPorId(...

    @PutMapping("/{id}")
    public ResponseEntity<Aluno> atualizar(
            @PathVariable Integer id,
            @RequestBody Aluno novosDados) {

        Optional<Aluno> alunoAtualizado = alunoService.atualizar(id, novosDados);

        if (alunoAtualizado.isPresent()) {
            return ResponseEntity.ok(alunoAtualizado.get());
        }

        return ResponseEntity.notFound().build();
    }
}
```

Postman

Atualizando um aluno com PUT

O método PUT representa uma atualização completa. Por isso, o cliente deve enviar todos os campos que podem ser atualizados, mesmo quando somente um deles tiver sido alterado.

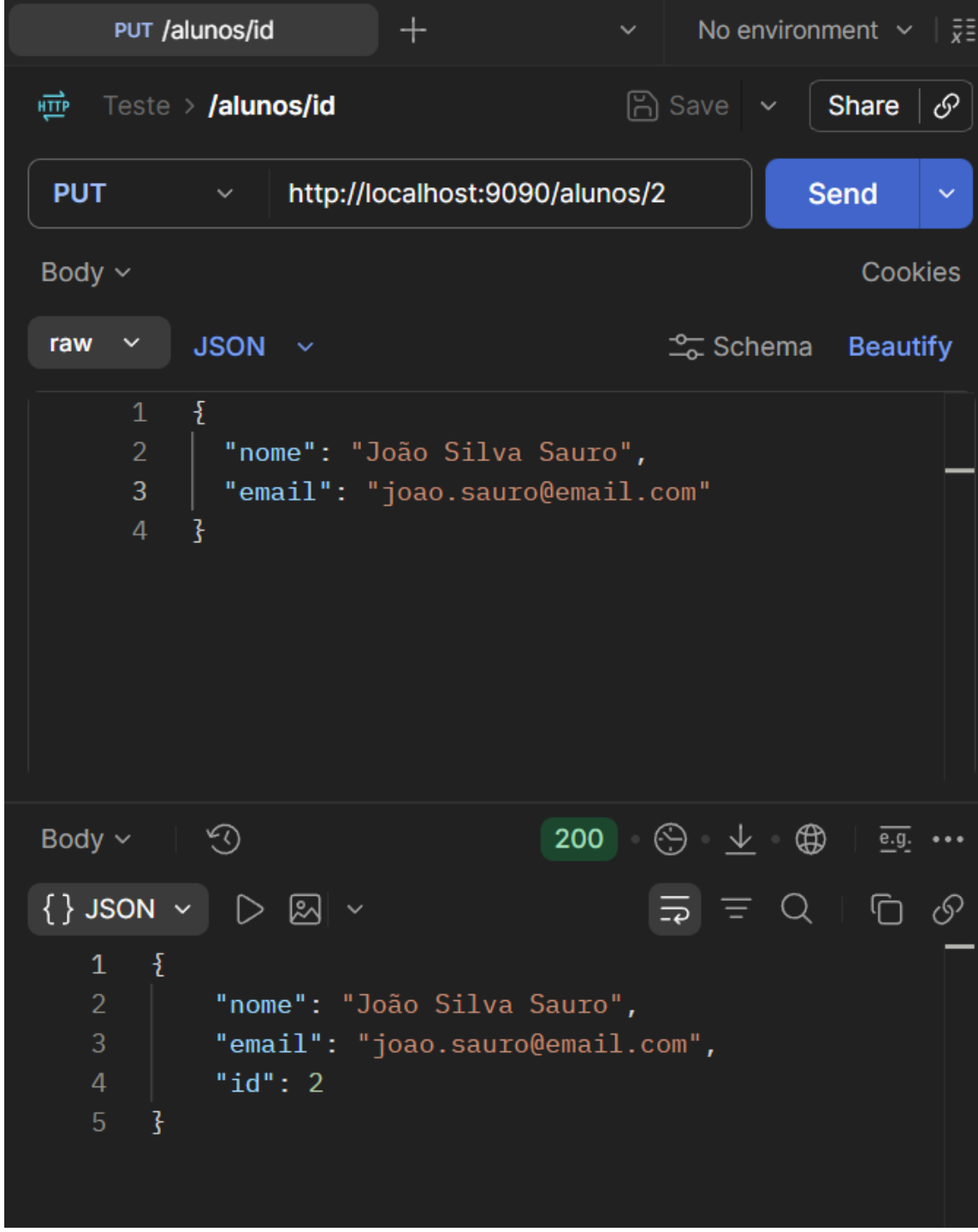
Realize os dois testes para verificar o comportamento da API.

RESPOSTAS POSSÍVEIS:

- 200 OK: o aluno foi encontrado e atualizado;
- 404 Not Found: não existe aluno com o ID informado.

Confirmando alteração:

- GET http://localhost:8080/alunos/id



Exclusão de dados

Criando a requisição DELETE

Antes de atualizar um aluno, a aplicação precisa verificar se o ID informado existe. Quando o registro é encontrado, os novos dados são aplicados ao objeto e enviados novamente ao Repository.

ALTERAÇÃO NO AlunoService:

- Adicione a importação
- Adicione o método

A operação de exclusão receberá o ID do aluno pelo endereço. Antes de remover o registro, o Service verificará se o aluno existe no banco.

```java
package br.com.senai.teste.service;

import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

import br.com.senai.teste.model.Aluno;
import br.com.senai.teste.repository.AlunoRepository;

@Service
public class AlunoService {

    private final AlunoRepository alunoRepository;

    public AlunoService(AlunoRepository alunoRepository) {
        this.alunoRepository = alunoRepository;
    }

    public Aluno cadastrar(Aluno aluno) {
        return alunoRepository.save(aluno);
    }

    public List<Aluno> listar() {
        return alunoRepository.findAll();
    }

    public Optional<Aluno> buscarPorId(Integer id) {
        return alunoRepository.findById(id);
    }

    public Optional<Aluno> atualizar(
            Integer id, Aluno novosDados) {

        Optional<Aluno> alunoEncontrado = alunoRepository.findById(id);

        if (alunoEncontrado.isEmpty()) {
            return Optional.empty();
        }

        Aluno aluno = alunoEncontrado.get();

        aluno.setNome(novosDados.getNome());
        aluno.setEmail(novosDados.getEmail());

        return Optional.of(alunoRepository.save(aluno));
    }

    public boolean excluir(Integer id) {

        if (!alunoRepository.existsById(id)) {
            return false;
        }

        alunoRepository.deleteById(id);
        return true;
    }
}
```

Exclusão de dados

Criando a requisição DELETE

ALTERAÇÃO NO AlunoController:

- Adicione a importação
- Adicione o método

O @DeleteMapping("/{id}") associa o método a uma requisição HTTP DELETE. O ID presente no endereço identifica qual aluno deverá ser removido.

O Controller recebe apenas o resultado da operação: true quando houve exclusão ou false quando o ID não foi encontrado.

RESPOSTAS POSSÍVEIS:

- 204 No Content: aluno excluído com sucesso;
- 404 Not Found: o ID informado não existe.

AlunoController.java

src > main > java > br > com > senai > teste > controller > AlunoController.java

```
17  import br.com.senai.teste.service.AlunoService;
18
19  @RestController
20  @RequestMapping("/alunos")
21  public class AlunoController {
22
23      private final AlunoService alunoService;
24
25      public AlunoController(AlunoService alunoService) {···
28
29      @PostMapping
30      public ResponseEntity<Aluno> cadastrar(···
39
40      @GetMapping
41      public ResponseEntity<List<Aluno>> listar() {···
47
48      @GetMapping("/{id}")
49      public ResponseEntity<Aluno> buscarPorId(···
60
61      @PutMapping("/{id}")
62      public ResponseEntity<Aluno> atualizar(···
74
75      @DeleteMapping("/{id}")
76      public ResponseEntity<Void> excluir(
77              @PathVariable Integer id) {
78
79          boolean excluido = alunoService.excluir(id);
80
81          if (excluido) {
82              return ResponseEntity.noContent().build();
83          }
84
85          return ResponseEntity.notFound().build();
86      }
87  }
```

Postman

Removendo um aluno com DELETE

A requisição DELETE utilizará o ID presente no endereço para identificar o aluno que será removido. Essa operação não precisa receber um JSON no corpo da requisição.

Antes de excluir, consulte um aluno existente:

- GET http://localhost:8080/alunos/1

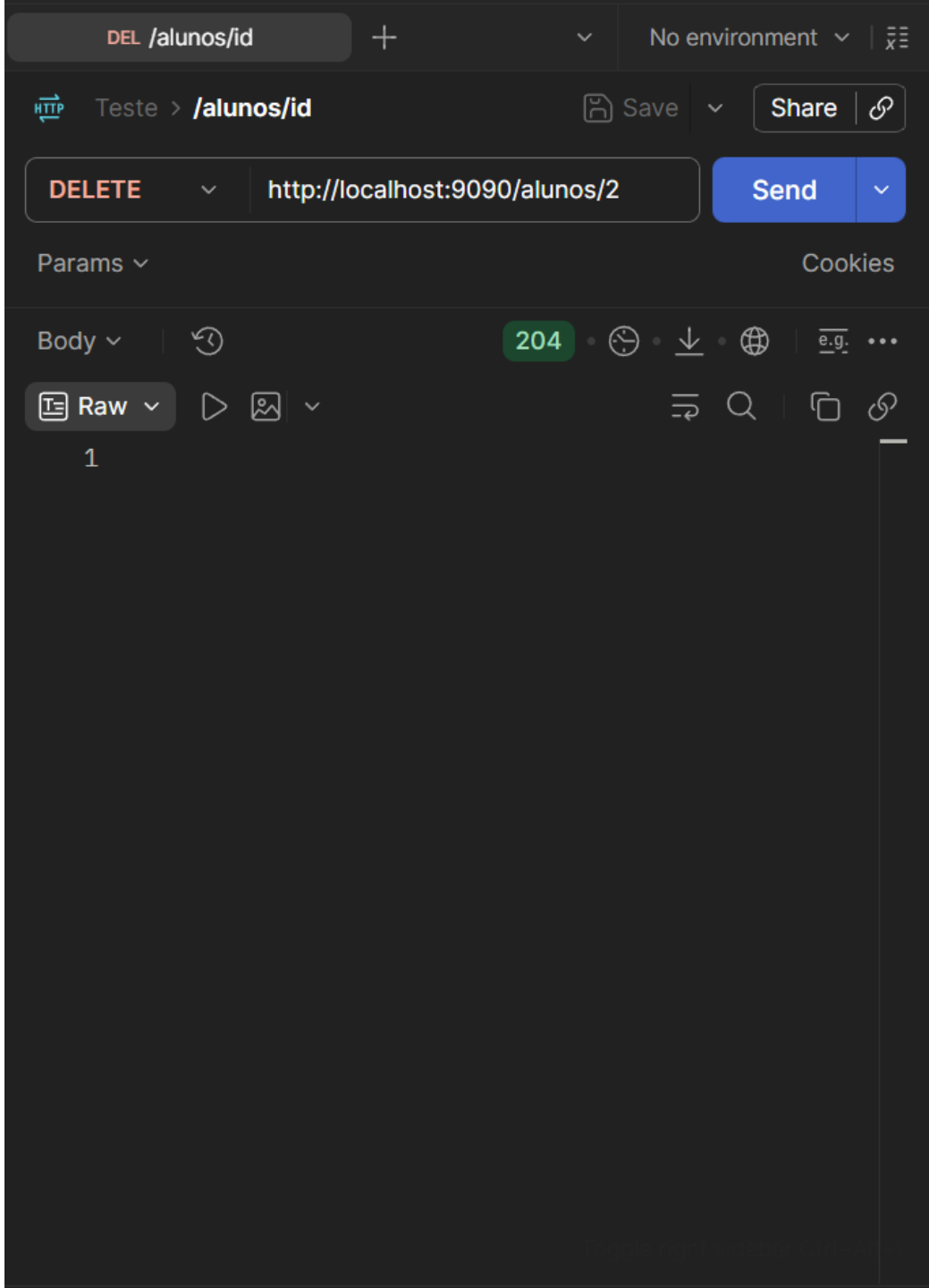
No Postman:

- selecione o método DELETE;
- informe o endereço com o mesmo ID;
- deixe o Body vazio;
- clique em Send.

RESULTADO ESPERADO

Se o aluno existir, a API deverá responder:

204 No Content



Resumo das operações

Rotas disponíveis na API de alunos

A API agora possui as operações básicas necessárias para cadastrar, consultar, atualizar e excluir alunos. Esse conjunto de operações é conhecido como CRUD.

Resumo



Operação	Método e rota	Repository	Resposta
Cadastrar	POST /alunos	save()	201 Created
Listar	GET /alunos	findAll()	200 OK
Buscar por ID	GET /alunos/{id}	findById()	200 ou 404
Atualizar	PUT /alunos/{id}	save()	200 ou 404
Excluir	DELETE /alunos/{id}	deleteById()	204 ou 404

O que é uma API?

Uma **API (Application Programming Interface)** é uma interface que permite que diferentes sistemas se comuniquem de forma organizada e padronizada. Em uma aplicação backend, ela define **quais operações podem ser solicitadas**, **como a requisição deve ser enviada** e **qual resposta será devolvida**.

No CRUD que desenvolvemos, criamos uma **API** capaz de receber requisições HTTP para **cadastrar, consultar, atualizar e excluir dados** do banco.

Saiba Mais!



A API funciona como uma porta de acesso controlada à aplicação. O cliente não acessa diretamente o banco de dados: ele envia uma requisição para um endpoint, e a aplicação decide como processá-la.

Termos importantes:
Endpoint: endereço disponibilizado pela API, como /alunos ou /produtos/5.
Método HTTP: indica a operação desejada, como GET, POST, PUT e DELETE.
Request: requisição enviada pelo cliente para a API.
Response: resposta devolvida pela API ao cliente.
JSON: formato muito utilizado para enviar e receber dados.
Status HTTP: código que informa o resultado da requisição, como 200 OK, 201 Created ou 404 Not Found.

Firjan SENAI